

ITM SKILLS UNIVERSITY

SCHOOL OF FUTURE TECH

Case Study Report

on

SpendWise - Personal Expense Tracker

(Backend Development – Node.js, Express.js, MongoDB & Socket.io)

NINAD NILESH DEODHARE

15009625180

SAM ALTMAN COHORT

Index:

- 1. Introduction to the Case Study**
- 2. Problem Statement / Case Background (Abstract)**
- 3. Problem Statement / Case Study Design**
- 4. Methods & Algorithms Technology Applied in the Case Study**
- 5. Case Study Implementation Details and Snapshots**
- 6. Case Study Results and Conclusion**
- 7. References**

1. INTRODUCTION TO CASE STUDY:

Managing personal finances effectively is a key challenge for individuals and families in modern daily life. Logging daily expenses manually on paper or offline spreadsheets often leads to untracked spending, budget overruns, and a lack of real-time awareness regarding individual or family financial health.

With the advancement of web technologies and asynchronous database management systems, personal finance tracking can be automated. An **Expense Tracker System** allows users to record daily income and expenditure, categorize spending (such as Groceries, Food, Rent, Entertainment), set monthly category budgets, receive real-time threshold alerts, and generate graphical financial reports.

This case study demonstrates the development of a production-ready, modular RESTful API and WebSocket system called **SpendWise** using **Node.js**, **Express.js**, **MongoDB (Mongoose ODM)**, **Socket.io**, and **Firebase Admin SDK**. The system handles user authentication, CRUD operations for transactions and budgets, real-time WebSocket budget threshold alerts, push notifications, and automated financial reporting.

The application features clean REST endpoints alongside an interactive web dashboard interface, ensuring high performance, scalability, and instant real-time user feedback.

2. PROBLEM STATEMENT / CASE BACKGROUND (ABSTRACT):

The purpose of this case study is to design and implement a scalable backend system for "**SpendWise**" that enables users to manage personal and family finances seamlessly.

The system provides functionality to:

- **User Authentication:** Secure user registration and login using JWT (JSON Web Tokens) and Firebase Authentication verification.
- **Transaction Management:** Full CRUD capability (Create, Read, Update, Delete) for income and expense transactions.
- **Category & Budget Configuration:** Assign spending limits per category on a monthly basis (YYYY-MM).
- **Real-Time Budget Alerts:** WebSockets using **Socket.io** to emit real-time overspending alerts to clients the moment a logged transaction exceeds a category's budget threshold.
- **Push Notifications:** **Firebase Cloud Messaging (FCM)** integration to send push notifications for overspending.
- **Financial Analytics & Forecasting:** Generating monthly pie-chart data breakdowns, category summaries, annual reports, and a 3-month moving average predictive spending forecast.

3. CASE STUDY DESIGN:

The SpendWise system is designed following a modular Model-View-Controller (MVC) architectural pattern:

```
Backend main project/
├── config/                # Database & Firebase connections
│   ├── db.js
│   └── firebase.js
├── controllers/          # Business logic layer
│   ├── authController.js
│   ├── transactionController.js
│   ├── budgetController.js
│   ├── categoryController.js
│   ├── reportController.js
│   └── notificationController.js
├── middleware/           # Authentication & validation layer
│   ├── auth.js
│   └── validate.js
├── models/               # Mongoose MongoDB schemas
│   ├── User.js
│   ├── Transaction.js
│   ├── Budget.js
│   └── Category.js
├── routes/               # Express REST API endpoints
├── utils/                # Socket.io event emitter helpers
│   └── socket.js
├── public/               # Web Dashboard frontend interface
│   ├── index.html
│   ├── style.css
│   └── app.js
├── server.js             # Main HTTP & Socket.io server entry point
├── package.json
└── README.md
```

Main Modules

- 1. Authentication Module (/api/auth):** Hashes passwords with bcrypt, manages user accounts, and returns JWT tokens.
- 2. Transaction Management Module (/api/transactions):** Logs expenses/incomes and calculates background category spent totals.
- 3. Budget & Real-Time Alert Module (/api/budgets):** Sets monthly limits and emits real-time Socket.io alerts when threshold is crossed.
- 4. Analytics & Forecast Module (/api/reports):** Calculates monthly summaries, category totals, yearly breakdowns, and moving average forecasts.

5. Firebase Push Notification Module (/api/notifications): Sends FCM push notifications to user devices.

4. METHOD AND ALGORITHMS APPLIED TO CASE STUDY:

The SpendWise system applies several advanced backend techniques:

1. MongoDB Aggregation Pipeline (\$match & \$group)

To compute total category spending for a user efficiently inside MongoDB:

```
const totalExpense = await Transaction.aggregate([
  {
    $match: {
      user: new mongoose.Types.ObjectId(userId.toString()),
      category: category,
      type: 'expense',
      date: { $gte: startOfMonth, $lte: endOfMonth }
    }
  },
  {
    $group: {
      _id: null,
      total: { $sum: '$amount' }
    }
  }
]);
```

2. Real-Time Socket.io Event Emission Algorithm

Whenever a transaction pushes spending over the limit, a WebSocket alert is emitted to the client's room:

```
if (spentAmount > budget.limit) {
  io.to(userId.toString()).emit('budget_alert', {
    budgetId: budget._id,
    category: budget.category,
    limit: budget.limit,
    spent: spentAmount,
    overspentBy: spentAmount - budget.limit,
    isExceeded: true,
    message: `ALERT: You have exceeded your budget for ${budget.category} by $${(spentAmount - budget.limit).toFixed(2)}!`
  });
}
```

3. Bcrypt Password Hashing

User passwords are salted and hashed using `bcryptjs` prior to database storage.

5. CASE STUDY IMPLEMENTATION DETAILS AND SNAPSHOTS:

The application is implemented in Node.js and Express.js with a MongoDB database.

Step 1: Server Setup (`server.js`)

```
const express = require('express');
const http = require('http');
const connectDB = require('./config/db');
const { initSocket } = require('./utils/socket');

connectDB();

const app = express();
const server = http.createServer(app);
initSocket(server);

app.use(express.json());
app.use(express.static('public'));

app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/transactions', require('./routes/transactionRoutes'));
app.use('/api/budgets', require('./routes/budgetRoutes'));
app.use('/api/reports', require('./routes/reportRoutes'));

server.listen(5000, () => {
  console.log('🚀 Server running on http://localhost:5000');
});
```

Step 2: Budget Mongoose Schema (`models/Budget.js`)

```
const mongoose = require('mongoose');

const BudgetSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  category: { type: String, required: true },
  limit: { type: Number, required: true },
  spent: { type: Number, default: 0 },
  month: { type: String, required: true }
}, { timestamps: true });

BudgetSchema.index({ user: 1, category: 1, month: 1 }, { unique: true });

module.exports = mongoose.model('Budget', BudgetSchema);
```

6. CASE STUDY RESULTS AND CONCLUSION:

The **SpendWise Expense Tracker Backend API** successfully demonstrates the design and deployment of a modern asynchronous Node.js, MongoDB, and WebSocket architecture.

Key Results Achieved:

- **Full RESTful API:** Clean CRUD operations for users, transactions, budgets, categories, reports, and notifications.
- **Real-Time Alert System:** Socket.io successfully emits overspending alerts instantly to clients.
- **Database Efficiency:** MongoDB indexing and aggregation pipelines compute financial metrics in sub-millisecond response times.
- **Interactive UI & Docs:** Swagger UI at `/api-docs` and an interactive web dashboard interface.

Future Improvements

- Automated SMS bank transaction parsing.
- Multi-currency exchange rate API integration.
- Machine-learning-based monthly expenditure predictions.

7. REFERENCES:

1. Node.js Official Documentation – <https://nodejs.org/docs>
2. Express.js API Reference – <https://expressjs.com>
3. MongoDB & Mongoose ODM Documentation – <https://mongoosejs.com>
4. Socket.io Real-time Application Framework – <https://socket.io/docs>
5. Firebase Admin SDK Guide – <https://firebase.google.com/docs/admin/setup>